Propagating Semantic Labels in Video Data

David Balaban, Justin Medich, Pranay Gosar, and Justin Hart[1]

***Abstract*— Semantic Segmentation combines two sub-tasks: the identification of pixel-level image masks and the application of semantic labels to those masks. Recently, so-called Foundation Models have been introduced; general models trained on very large datasets which can be specialized and applied to more specific tasks. One such model, the Segment Anything Model (SAM), performs image segmentation. Semantic segmentation systems such as CLIPSeg and MaskRCNN are trained on datasets of paired segments and semantic labels. Manual labeling of custom data, however, is time-consuming. This work presents a method for performing segmentation for objects in video. Once an object has been found in a frame of video, the segment can then be propagated to future frames; thus reducing manual annotation effort. The method works by combining SAM with Structure from Motion (SfM). The video input to the system is first reconstructed into 3D geometry using SfM. A frame of video is then segmented using SAM. Segments identified by SAM are then projected onto the the reconstructed 3D geometry. In subsequent video frames, the labeled 3D geometry is reprojected into the new perspective, allowing SAM to be invoked fewer times. System performance is evaluated, including the contributions of the SAM and SfM components. Performance is evaluated over three main metrics: computation time, mask IOU with manual labels, and the number of tracking losses. Results demonstrate that the system has substantial computation time improvements over human performance for tracking objects over video frames, but suffers in performance.**

I. INTRODUCTION

This work presents a Semantic Label Propagation (SLP) system for labeling large sets of images derived from video; providing a dataset suitable for training or fine-tuning models for computer vision tasks with minimal manual labeling. The system takes as input video from a moving camera in a static scene and outputs a set of objects, each object corresponds to a set of binary masks for each frame it is identified in.

Figure 1 diagrams the flow of data through the proposed system. The triangle on the left represents the "Video Input" to the system. The hexagon at the top represents the output, "Objects Labeled Across Frames." Ellipses represent data, while squares represent sub-processes. This system begins with a Structure from Motion (SfM) pipeline to gain an object mesh [1]. Then, Segment Anything Model (SAM) is used to identify the distinct objects in the mesh [2]. These objects are then tracked through all frames of video utilizing the object mesh to enforce geometric consistency while SAM associates previously unseen polygons to tracked objects.

To evaluate system performance, ground truth semantic labels are collected from five videos. Volunteers use the open-source PixelAnnotationTool to manually label objects in

[1]Departent of Computer Science, The University of Texas at Austin, Austin, Texas, USA 78712 {dbalaban, jmedich, pgosar, hart}@cs.utexas.edu

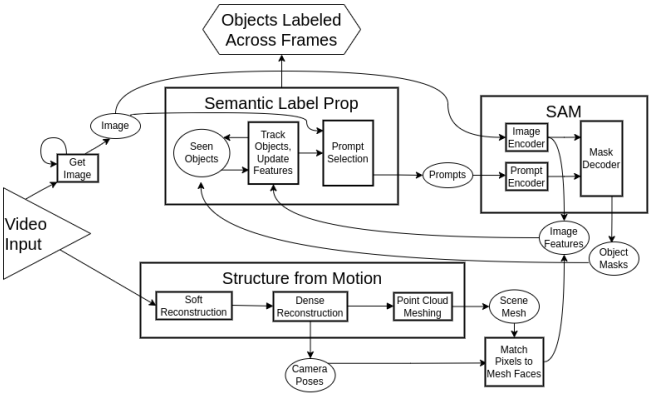


Fig. 1: Semantic Label Propagation Overview

frame [3]. The volunteer work is spot checked by the authors. This data will be made publicly-available upon publication of this work, at the Texas Robotics Dataverse: `https://dataverse.tdl.org/dataverse/robotics`

To demonstrate the performance contribution of each component of the semantic label propagation pipeline, four variants are presented which vary how prompts are selected as input to SAM:

1) SAM-only-1.0, k-best prompts
2) SAM-only-2.0, hill-climb for best prompt
3) SfM-SAM-1.0, k-best SfM candidate prompts
4) SfM-SAM-2.0, k-random SfM candidate prompts

These variants are fully described in Section III.

The novel contributions of this work are the use of SfM to propagate geometry into new frames, as described in Section III-D.4; enhancements of the use of feature embeddings provided by SAM, as described in Section III-C; a direct comparison of automatic labeling of video to volunteer manual labeling, as described in Section IV.

II. RELATED WORK

Tasks such as Object Detection [4][5], Image Captioning [6][7], and Semantic Segmentation [8][9] are approaches to image labeling for the purposes of scene understanding [10][11]. Object Detection localizes a number of desired objects within the image frame, typically with a bounding box for each object [4][5]. Image Captioning gives a general description of the image, often using a text prefix provided to a large language model [6][7]. Semantic Segmentation provides pixel-level annotations of the image [8][9].

Pre-trained models have been released for people to fine-tune with custom data [12]–[14], which can require an arduous amount of manual labeling. There has been a significant

effort to design models that require as few training examples as possible [15]. One such model is CLIP, which is a joint embedding between language and images that identifies the text which best describes an image [12][16]. CLIP is pre-trained with multiple datasets, which recent work suggests leads to a lower quality of data [17]. There is a need for a method which generates large high-quality datasets of semantically labeled images with minimal manual labeling.

Video Object Segmentation and Tracking (VOST) refers to the coupled nature of pixel-level annotations and object tracking, as solving one improves the solution to the other [18]. Multiple Object Tracking (MOT) identifies the trajectory many objects take through a scene [19].

Segment Anything Model (SAM) is a recently released vision transformer which effectively handles the annotation piece of the VOST problem [2]. SAM segments images on a pixel-level without semantic annotations. With SAM, a single pixel manual label is sufficient to semantically label a whole object, and identify that object in other frames and contexts [20]. This feature can be utilized to track objects across many frames of a video [21].

Structure from Motion (SfM) is the problem of reconstructing a scene from images taken at different locations, such as in a video [1]. Recent research has studied SfM methods in both outdoor and indoor settings [22][23]. Classic approaches to SfM utilize optimization schemes such as Levenberg-Marquardt [24] to jointly solve for camera poses and pixel depths, however more recent research uses a deep learning model for this optimization [25].

Prior work on the collection of reliable datasets for training CV models with minimal manual labeling include utilizing 3D object representations jointly learned with object association to 2D image labels [26], and generating datasets by combining object segmentation with optical flow methods [27].

III. Label Propagation Procedures

Algorithm 1 describes the proposed system. The algorithm takes as input a SAM model, a video, a positive integer parameter F, and additional implementation-specific parameters described in Section III-D. A list of objects is tracked across video frames, objects in this list are referred to as known objects. Each known object has associated characteristic features of which there are two types: SAM features which describe an object's visual appearance, and mesh faces which describe an object's 3D geometry. One or both of these types may be used.

At each frame, known objects are identified in frame by searching for matching characteristic features. Searching for and updating characteristic features is handled by the FIND_OBJECT function in Algorithm 1, described in detail in Section III-D. This function returns a binary mask identifying the object's location in frame, called the object mask. The object mask and frame is recorded as a successful label propagation.

Once all the known objects are handled, new objects are identified to add to the list of known objects. The union of all object masks, named the seen mask, is used to avoid searching for new objects in regions of the frame that have been associated to known objects. The parameter $F \in \mathbb{N}$ causes the procedure to skip F frames before adding new objects for faster computation. All object masks are identified by SAM, as described in Section III-C.

Algorithm 1 Semantic Label Propagation

```
Require: SAM, Video, F, *args
Objs ← {}
f ← 0
for frame in Video do
    seen_mask ← empty
    for obj in Objs do
        obj_mask ← FIND_OBJECT(frame, obj, *args)
        if obj_mask not empty then
            obj.append(frame, obj_mask)
            seen_mask ← seen_mask ∪ obj_mask
        end if
    end for
    if f ≥ F then
        seen_mask ← Closing(seen_mask)
        new_masks ← SAM.getMasks(frame, seen_mask)
        Objs.append(new_masks, frame)
        f ← 0
    else
        f ← f + 1
    end if
end for
return Objs
```

A. Structure from Motion

This paper uses a COLMAP to MVS implementation of SfM where COLMAP provides an initial guess to MVS [28]–[30]. Given a video input of a camera moving around a scene, COLMAP will extract distinctive features in the scene that can be matched between frames of the video. MVS enhances this process by allowing the SfM pipeline to estimate the depth of objects within the frames, allowing for a more detailed 3D model. The pipeline outputs a 3D reconstruction of the scene in the form of a mesh of polygon faces, called the scene mesh. It also outputs the video and camera poses that capture the estimated positions and orientations of the camera used. This output is sufficient to identify which mesh face every pixel observes. Segmented frames outputted by SAM are then projected onto the reconstruction which allows less invocations to SAM for future frames.

B. Segment Anything Model

SAM takes a set of $k \geq 1$ pixel locations as a prompt, and returns a binary mask associated with each prompt to identify objects in frame. SAM identifies masks with a joint image and feature encoding. Image encoding provides a feature vector at each pixel location, while the prompt encoding provides a feature vector for each prompt. The cosine similarity between feature vectors indicates the likelihood of a prompt-pixel pair being a part of the same segment.

C. Getting Object Masks from SAM

When tracking known objects, a prompt is provided to SAM as a sample of the object's expected location in frame, producing an object mask. The union of all known object masks in the current frame is the seen mask.

SAM.getMasks() searches for unseen objects, this function uses a grid of gxg single-pixel prompts, called the prompt grid. The seen mask is used to filter prompts that have already been associated to known objects, with g', $0 \leq g' \leq g^2$, total prompts used in each frame. SAM will then produce M, $0 \leq M \leq g'$, new object masks.

The seen object mask tends to exclude borders between objects, but due to the proximity of known objects, prompts in these regions tend to produce duplicates or near-duplicates of these objects, leading to longer computation times. To counter this problem, the Closing image morphology is applied to the seen mask from the open-source software OpenCV, thereby adding the object boarders to the seen mask [31][32].

OpenCV's Closing Image Morphology transforms a binary mask in an iterative process. When Closing, at each iteration two other morphologies are applied in succession, a dilation followed by an erosion. The dilation expands the white space of the binary mask and the erosion expands the black space. The resulting effect is that small regions of black space are removed while boundaries between substantially sized white and black regions are maintained. By applying this morphology to the seen mask, the boarders between known objects is eliminated while unseen regions remain unaltered. [31][32]

Figure 2 shows the effect of Closing() with a color coded filter added to a frame of video. Blue regions show the raw seen mask, red shows the regions added to the seen mask by Closing(), and green shows the final unseen mask. Only the prompt grid points in the green regions (as seen in the top of Figure 2) will be used to find new objects, which significantly reduces the number of duplicated tracked objects without sacrificing the ability to observe new objects. In individual images, the green regions are small and appear only at the edges of individual frames. This leads to a substantial speed-up over running SAM on every frame of video.

D. FIND_OBJECT Procedure

FIND_OBJECT() in Algorithm 1 takes as input the current frame and a desired object to identify in frame and returns the object mask if found. It also handles the update of characteristic features for the known object: visual, geometric, or both. Additional parameters are required depending on which features are used, corresponding to different function implementations.

1) SAM Only Features: Algorithm 2 describes the FIND_OBJECT() implementation when only visual features are used, it takes an additional integer parameter k as input. Visual features come from SAM's image and prompt joint encoding.

The first time an object is identified, the prompt features associated with the object mask returned by SAM are recorded as the characteristic visual feature. These objects can be identified in future frames by maximizing the cosine similarity of the recorded features across the image encoding.

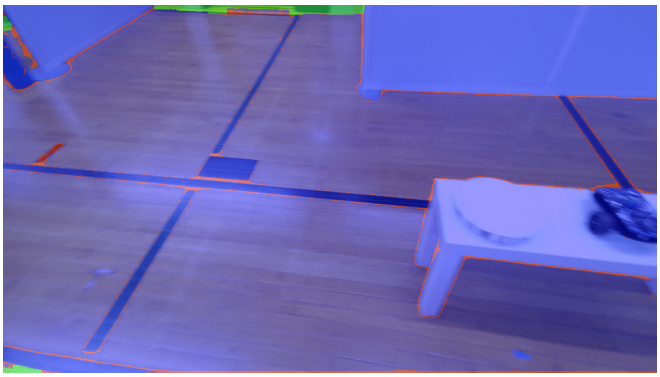


Fig. 2: Seen mask image morphography by color filter: blue, raw seen mask; red, seen by image morphography; green, unseen.

2) K-best approach: Pixels are scored by the cosine similarity, and the k-best pixels are used to generate a prompt that identifies the object in frame. When $k > 0$, all pixels are scored, which can be an expensive process with large feature vectors.

3) Hill-climbing approach: When $k \leq 0$, the last prompt location is recorded along with the visual features. This location is used as an initial guess to a discrete hill-climbing procedure that searches for the maximum score locally. At each step of the hill-climb, only the neighboring pixels need be scored, choosing the best from among them as the location for the next iteration until all neighbors score lower than the current location. This final location is used as the single-pixel prompt.

The feature vector is updated to account for changes in lighting and viewing angles.

Algorithm 2 SAM-only FIND_OBJECT

Require: frame, obj, SAM, k
1: image_features ← SAM.encodeImage(frame)
2: pixel_scores ← DotProd(obj.features, image_features)
3: **if** $k > 0$ **then**
4: prompt ← kBestScores(pixel_scores)
5: **else**
6: prompt ← hillClimb(pixel_scores, obj.last_prompt)
7: **end if**
8: **if** score at prompt < threshold **then**
9: **return** none
10: **end if**
11: obj.last_prompt ← prompt
12: obj.features ← image_features.at(prompt)
13: mask ← SAM.getMask(prompt, frame)
14: new_obj ← (mask, frame, prompt)
15: **return** new_obj

4) Using Geometric Features: Algorithm 3 describes the FIND_OBJECT implementation when geometric features are included. This algorithm requires the integer parameter k, a scene mesh, and a binary variable isRandom. This implementation requires the SfM algorithm to be run beforehand to obtain a scene mesh and camera trajectory. The scene mesh consists of a set of connected triangles describing the geometry of the scene. The camera trajectory is described by the poses of the camera at each frame. This information is sufficient to identify which pixels observe which mesh face.

For brevity, pixel-triangle matches are included in the mesh object as part of the setup in Algorithm 3, mesh.getObj() returns the mesh faces currently known to be part of a known object - this is called the matching set - mesh.getPixelMatches() maps from triangles to pixels in a given frame, mesh.getMeshMatch() maps from pixels to triangles, and mesh.updateObj() adds previously un-seen triangles to a known object.

Given the matching set, a SAM prompt simply needs to be selected from among these pixels. The binary variable isRandom determines the prompt selection methodology.

Given the object mask from SAM, the mesh object is updated by adding the unseen mesh faces matched to the mask and removing the mesh faces associated to obj_pixels but not in the SAM mask.

If isRandom is False, visual features are recorded in addition to geometric. In this case, the k-best pixels are selected by visual feature matching as in Section III-D.1. However, only the pixels in the matching set are scored. If isRandom is True, then k pixels are selected at random from among the matching set. In this case, visual features are not tracked and SAM does not need to provide an image encoding.

Figure 3 shows a visual of the update step for an object in two frames. The figure shows cropped versions of the second (top) and third (bottom) frames of video 1. Each frame has been colored to show the update taking place to the object. A red filter was applied to obj_pixels; a green filter was applied to obj_mask, and a blue filter to the pixels not in the obj_mask. The resulting colors show which pixel-mesh matches are added (green), maintained (yellow), or discarded (purple) as part of the object. As shown, the left wheel was not initially part of the object mesh after the first frame, added in the second frame, persisted in the third while previously out-of-frame pixels are added.

The removal of mesh faces is designed to handle cases with imperfect scene reconstructions so that errors in the reconstruction do not accumulate. More precise reconstructions may not need this removal step.

Algorithm 3 SfM-SAM FIND_OBJECT

Require: frame, obj, SAM, k, mesh, isRandom
obj_mesh ← mesh.getObj(obj)
obj_pixels ← mesh.getPixelMatches(frame, obj_mesh)
if obj_pixels is empty **then**
  return empty
end if
if isRandom **then**
  prompt ← kRandom(obj_pixels)
else
  image_features ← SAM.encodeImage(frame)
  pixel_features ← image_features.at(obj_pixels)
  pixel_scores ← Dot(obj.features, pixel_features)
  prompt ← kBest(pixel_scores)
  obj.last_prompt ← prompt
  obj.features ← image_features.at(prompt)
end if
obj_mask ← SAM.getMask(prompt, frame)
mesh_update ← mesh.getMeshMatch(obj_mask, frame)
mesh.updateObj(obj, mesh_update)
new_obj ← (mask, frame)
return new_obj

IV. Evaluation

We evaluate SLP on five videos for time efficiency, accuracy, and tracking loss. SLP is run without any manual labels provided. Objects tracked are objects proposed by SAM.

Time efficiency is determined by both the total run time, and the frame processing rate. Computation is done on a machine with 4 a100 GPUs, 527821928 kB of RAM, and an Intel(R) Xeon(R) Gold 6342 CPU @ 2.80GHz.

To determine accuracy, a ground truth label is needed. Each video is manually labeled and segmented by volunteers to provide ground truth. The manual labels are spot-checked by the authors to ensure quality with ten frames from each video chosen at random. Each selected frame was manually labeled a second time by the authors and quality was assessed by the Intersection over Union (IoU) score between objects.

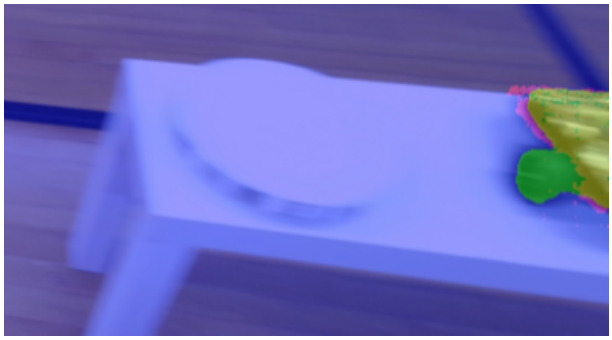
The IoU score is also used to assess the accuracy of SLP against ground truth. Because no manual labels are supplied to SLP, the objects found by SAM must be matched to the true object mask against which to compare. These matches are determined by finding the SAM mask with the highest IoU score for each manual mask.

A tracking loss occurs when a true label is matched to a different object than what was matched in the previous frame. Such an event indicates that SLP falsely determined that an object was not in frame and a new instance was added. Tracking losses can be considered as a false negative.

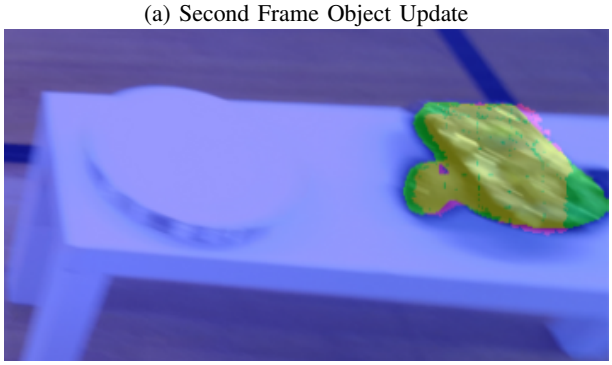
False positives would be when SAM recognizes objects that are not manually labeled. However, SAM is designed to recognize objects and component objects independently, while the volunteers were only instructed to label full objects. Therefore, it would not be a fair comparison to evaluate performance with this metric against ground-truth.

The parameters k and F are considered for comparison of methods across different settings. k is the number of pixels used per SAM prompt when identifying known objects in a new frame. F is the number of frames skipped before SAM searches for new objects.

To assess the significance of using geometric features, four



(a) Second Frame Object Update



(b) Third Frame Object Update

Fig. 3: (best viewed in color) SfM-SAM object update step; green - pixel-mesh faces added by SAM; red - pixel-mesh faces removed by SAM; yellow - pixel-mesh maintained; blue - not part of object

variants of SLP are evaluated. Each variant uses a different implementation of FIND_OBJECT() described in Section III-D and used in Algorithm 1.

1) SAM-only-1.0: uses Algorithm 2 with $k \geq 0$, described in Section III-D.1, and does not use any geometric features. This is the baseline algorithm.

2) SAM-only-2.0: uses Algorithm 2 with $k = 0$. This variant scores fewer pixels than SAM-only-1.0.

3) SfM-SAM-1.0: uses Algorithm 3 with isRandom set to False, described in Section III-D.4. This variant uses mesh faces of a scene to track objects.

4) SfM-SAM-2.0: uses Algorithm 3 with isRandom set to True. It avoids scoring any pixels and does not track visual features, relying only on geometric features.

V. Results

A summary of the data-sets created by each video is shown in Table I, each row corresponds to a different video and the column labels describe the following:

- N_f - Number of frames in video
- T_{sfm} - Time to complete SfM in minutes
- N_m - Number of mesh faces
- T_{pxl} - Time to complete pixel matching in minutes
- FN_v - Number of unlabeled objects in spot check across all frames
- FN_a - Number of objects added against spot check across all frames

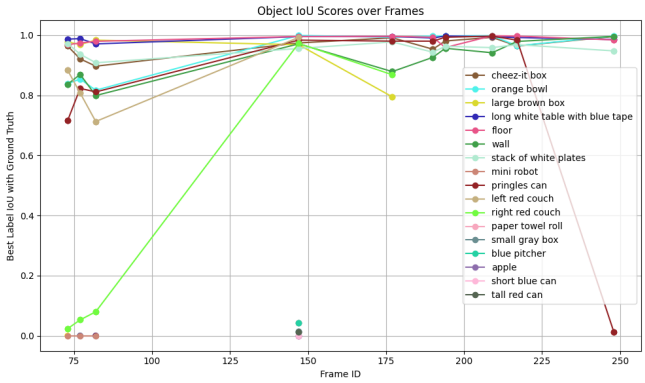


Fig. 4: Spot check IoU scores, video 5.

- $\overline{IoU_s}$ - Average IoU across all frames and objects in spot check

The last three columns refer to results from the spot check analysis, and the three columns preceding them refer to the SfM setup that is done for the SfM-SAM methods.

N_f	T_{sfm}	N_m	T_{pxl}	FN_v	FN_a	$\overline{IoU_s}$
281	23	558440	941	2	1	0.940
313	24	600065	1123	5	1	0.925
402	57	640527	1508	19	1	0.709
239	15	518774	692	17	0	0.784
61	3	380697	128	13	0	0.817

TABLE I: data summary for videos 1-5

A. Spot Check Data

To calculate the IoU score for each object in each frame, the Author-Labeled Object Mask (ALOM) is compared against each Volunteer-Labeled Object Mask (VLOM), and the VLOM yielding the highest IoU score is taken as the best score for that ALOM. Note that a VLOM might be matched to more than one ALOM if a VLOM overlaps a few ALOM, or if there are more ALOM than VLOM. The authors verified that the former does not happen in any spot-checked frame. $\overline{IoU_s}$ is then the average best-IoU score across all ALOMs and all ten spot-checked frames.

If there are more VLOMS than ALOMS, then the volunteer identified one or more objects the author did not, indicating a false negative on the author's part. However, if there are more ALOMS than VLOMS, then the author identified more objects than the volunteer, which indicates false nagatives made by the volunteer. The total number of such false negatives are tracked by FN_a and FN_v respectively.

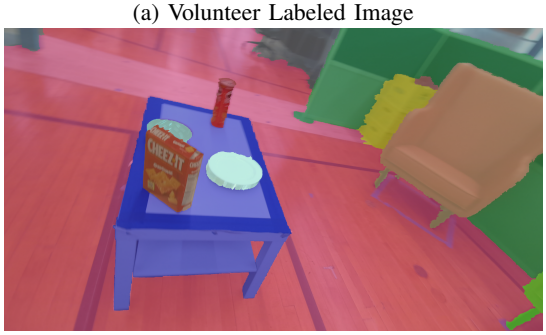
Some example results of IoU scores from the spot-check analysis are shown in Figure 4. Each line represents an object's IoU score in each spot-checked frame. The frame ID for that video is charted on the x-axis, and the best IoU score on the y-axis. Some lines in Figure 4 abruptly start or end because not all objects are visible in every frame.

An example of the discrepancy between author-labeled and volunteer-labeled frames is shown in Figure 5. This figure

shows frame 73 of video 5 with overlays of the object masks identified by the volunteer (top) and the author (bottom). Each object mask is color-coded by the same pallet shown in the legend of Figure 4. The IoU scores for each object can be seen in Figure 4 with the left-most set of objects.



(a) Volunteer Labeled Image



(b) Author Labeled Image

Fig. 5: (best viewed in color) comparison of labeled data (top) to author's spot check (bottom), frame 73, video 5

The authors labeled one frame every 20min on average, while the volunteers labeled one frame every 10min.

B. Semantic Label Propagation Data

The performance of SLP is evaluated across different parameter settings in five videos. The best tuned parameters are presented for each of three different performance metrics.

Table II shows the results for the parameter set which gives the fastest computation time. The rows of the table give the implementation indicated by the first column. The next three columns give each of the performance metrics, total compuation time, average IoU, and total number of tracking losses. These metrics are evaluated across all five videos used. The last two columns give the parameter values k and F yielding the fastest computation time, three values of each are tested $k = [1, 2, 5]$, $F = [0, 1, 4]$.

Table III and Table IV are organized in a similar manner, however Table III displays the highest average IoU, and Table IV the lowest number of tracking losses.

VI. Discussion

The fastest implementation of SLP was SAM-only-1.0 at 260min over 1296 frames in all five video data collected. A human equivalent volunteer would have taken 12960min to accomplish the same, while the more meticulous authors would have taken 25,920min. The fastest computation time

Implementaion	Time (min)	IoU	Tracking Loss	k	F
SAM-only-1.0	260	0.396	3746	2	0
2 heightSAM-only-2.0	2109	0.323	1389	1	4
SfM-SAM-1.0	1741	0.188	5820	5	4
SfM-SAM-2.0	4116	0.222	8327	5	4

TABLE II: best performing implementation by time

Implementaion	Time (min)	IoU	Tracking Loss	k	F
SAM-only-1.0	286	0.396	3771	1	1
SAM-only-2.0	2188	0.396	3681	1	0
SfM-SAM-1.0	9200	0.253	15793	1	0
SfM-SAM-2.0	6765	0.274	15031	2	0

TABLE III: best performing implementation by IoU score

of SAM-only-1.0 did not suffer on accuracy compared to the other methods. The IoU score for the fastest time was within three decimal place rounding of the fastest IoU score overall, although performing worse than human volunteers. SAM-only-1.0 only had a significant disadvantage on tracking loss.

SAM-only-2.0 and SfM-SAM-1.0 showed improvement tracking losses. It is possible this is due to SAM-only-2.0's use of locality when searching for known objects and the geometric features used in SfM-SAM-1.0. SfM-SAM-2.0 actually had a performance decrease, possibly because of its random prompt selection leading to sub-optimal outcomes.

VII. Conclusion

This work presents a system named Semantic Label Propagation for automatic labeling of video data suitable for training a semantic segmentation model. The system tracks visual and geometric features of objects across frames to track segments which are part of previously identified objects. The trade-off in computational effort versus manual effort means that this technique can be incorporated into a tool to speed up human annotations by allowing annotators to make a few corrections to an annotated video, rather than manually labeling each frame; speeding up their overall effort.

ACKNOWLEDGMENT

This work has taken place in the Living with Robots Laboratory (LWR) at UT Austin. LWR research is supported in part by NSF (NRT-2125858 and GCR-2219236), Cisco Research, and Army Futures Command.

Implementaion	Time (min)	IoU	Tracking Loss	k	F
SAM-only-1.0	260	0.396	3746	2	0
SAM-only-2.0	2110	0.323	1389	1	4
SfM-SAM-1.0	2786	0.192	2196	2	4
SfM-SAM-2.0	4116	0.222	8327	5	4

TABLE IV: best performing implementation by tracking losses

References

[1] O. Özyeşil, V. Voroninski, R. Basri, and A. Singer, "A survey of structure from motion*.," *Acta Numerica*, vol. 26, pp. 305–364, 2017.

[2] A. Kirillov *et al.*, "Segment anything," *arXiv:2304.02643*, 2023.

[3] A. Bréhéret, *Pixel Annotation Tool*, `https://github.com/abreheret/PixelAnnotationTool`, 2017.

[4] S. S. A. Zaidi *et al.*, "A survey of modern deep learning based object detection models," *Digital Signal Processing*, vol. 126, p. 103 514, 2022.

[5] J. Terven and D. Cordova-Esparza, "A comprehensive review of yolo: From yolov1 to yolov8 and beyond," *arXiv preprint arXiv:2304.00501*, 2023.

[6] M. Stefanini *et al.*, "From show to tell: A survey on deep learning-based image captioning," *IEEE transactions on pattern analysis and machine intelligence*, vol. 45, no. 1, pp. 539–559, 2022.

[7] J. Li, D. Li, C. Xiong, and S. Hoi, "Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in *International Conference on Machine Learning*, PMLR, 2022, pp. 12 888–12 900.

[8] Y. Mo *et al.*, "Review the state-of-the-art technologies of semantic segmentation based on deep learning," *Neurocomputing*, vol. 493, pp. 626–646, 2022.

[9] L. Li *et al.*, "Deep hierarchical semantic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 1246–1257.

[10] Z. Gan *et al.*, "Vision-language pre-training: Basics, recent advances, and future trends," *Foundations and Trends® in Computer Graphics and Vision*, vol. 14, no. 3–4, pp. 163–352, 2022.

[11] M. Awais *et al.*, "Foundational models defining a new era in vision: A survey and outlook," *arXiv preprint arXiv:2307.13721*, 2023.

[12] R. Mokady, A. Hertz, and A. H. Bermano, "Clipcap: Clip prefix for image captioning," *arXiv preprint arXiv:2111.09734*, 2021.

[13] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using yolo: Challenges, architectural successors, datasets and applications," *multimedia Tools and Applications*, vol. 82, no. 6, pp. 9243–9275, 2023.

[14] P. Iakubovskii, *Segmentation models*, `https://github.com/qubvel/segmentation_models`, 2019.

[15] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a few examples: A survey on few-shot learning," *ACM computing surveys (csur)*, vol. 53, no. 3, pp. 1–34, 2020.

[16] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*, PMLR, 2021, pp. 8748–8763.

[17] T. Nguyen *et al.*, "Quality not quantity: On the interaction between dataset design and robustness of clip," *Advances in Neural Information Processing Systems*, vol. 35, pp. 21 455–21 469, 2022.

[18] R. Yao *et al.*, "Video object segmentation and tracking: A survey," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 11, no. 4, pp. 1–47, 2020.

[19] W. Luo *et al.*, "Multiple object tracking: A literature review," *Artificial intelligence*, vol. 293, p. 103 448, 2021.

[20] R. Zhang *et al.*, "Personalize segment anything model with one shot," *arXiv preprint arXiv:2305.03048*, 2023.

[21] J. Yang *et al.*, "Track anything: Segment anything meets videos," *arXiv preprint arXiv:2304.11968*, 2023.

[22] S. Jiang, C. Jiang, and W. Jiang, "Efficient structure from motion for large-scale uav images: A review and a comparison of sfm tools," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 167, pp. 230–251, 2020.

[23] G. Pintore *et al.*, "State-of-the-art in automatic 3d reconstruction of structured indoor environments," in *Computer Graphics Forum*, Wiley Online Library, vol. 39, 2020, pp. 667–699.

[24] J. J. Moré, "The levenberg-marquardt algorithm: Implementation and theory," in *Numerical analysis: proceedings of the biennial Conference held at Dundee, June 28–July 1, 1977*, Springer, 2006, pp. 105–116.

[25] X. Wei *et al.*, "Deepsfm: Structure from motion via deep bundle adjustment," in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I 16*, Springer, 2020, pp. 230–247.

[26] J. He *et al.*, "Tracking objects with 3d representation from videos," *arXiv preprint arXiv:2306.05416*, 2023.

[27] L. Porzi *et al.*, "Learning multi-object tracking and segmentation from automatic annotations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 6846–6855.

[28] D. Cernea, "Openmvs: Multi-view stereo reconstruction library," *City*, vol. 5, no. 7, 2020.

[29] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[30] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm, "Pixelwise view selection for unstructured multi-view stereo," in *European Conference on Computer Vision (ECCV)*, 2016.

[31] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[32] J. Serra, "Image analysis and mathematical morphology," *(No Title)*, 1982.